

Experiment 1: Printing Even Numbers Using Loops and Array

Experiment No: 01

Experiment Name: Java Loop and Array Practice

Task Name: Print Even Numbers Using while and do-while Loops and Array

Objective:

1. Take starting and ending numbers from the user.
2. Identify all even numbers in the specified range.
3. Store the even numbers in an array.
4. Demonstrate the use of different loops (while and do-while) to display results.

Problem Analysis:

The goal is to find all even numbers within a range specified by the user and store them in an array. The challenge is to efficiently check each number and display the results. Using an array allows storing and accessing the numbers easily for further processing.

Input-Process-Output:

- **Input:** Start number, End number
- **Process:** Loop through the numbers, check for even numbers, store them in an array
- **Output:** Display the array of even numbers

Algorithm:

1. Take start and end numbers from the user.
2. Create an array or ArrayList to store even numbers.
3. Use a while loop from start to end to check each number.
4. If a number is even, add it to the array.
5. Display the array after the loop ends.
6. Repeat the process using a do-while loop.

Source Code:

```
EvenNumbers.java > EvenNumbers > main(String[])
1  import java.util.Scanner;
2  import java.util.ArrayList;
3
4  public class EvenNumbers {
5      Run | Debug
6      public static void main(String[] args) {
7          Scanner sc = new Scanner(System.in);
8          System.out.print("Enter starting number: ");
9          int start = sc.nextInt();
10         System.out.print("Enter ending number: ");
11         int end = sc.nextInt();
12
13         ArrayList<Integer> evenNumbers = new ArrayList<>();
14         int i = start;
15
16         // Using while loop
17         while (i <= end) {
18             if (i % 2 == 0) {
19                 evenNumbers.add(i);
20             }
21             i++;
22         }
23         System.out.println("Even numbers using while loop: " + evenNumbers);
24         // Using do-while loop
25         evenNumbers.clear();
26         i = start;
27         do {
28             if (i % 2 == 0) {
29                 evenNumbers.add(i);
30             }
31             i++;
32         } while (i <= end);
33
34         System.out.println("Even numbers using do-while loop: " + evenNumbers);
35         sc.close();
36     }
37 }
```

Output:

```
PS C:\Users\dell\Downloads\OneDrive\Documents\Shariar's Folder\2nd SEMESTER> java -cp bin EvenNumbers
Enter starting number: 1
Enter ending number: 20
Even numbers using while loop: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
Even numbers using do-while loop: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

Discussion:

This program demonstrates how to identify even numbers using loops and store them in an array. The difference between while and do-while loops is that do-while executes the first iteration unconditionally. Using an ArrayList allows dynamic storage of numbers.

Experiment 2: Storing and Printing Prime Numbers in an Array

Experiment No: 02

Experiment Name: Java Array and Prime Numbers

Task Name: Store and Print Prime Numbers in an Array

Objective:

1. Take starting and ending numbers from the user.
2. Identify prime numbers within the specified range.
3. Store the prime numbers in an array.
4. Display the prime numbers from the array.

Problem Analysis:

A prime number is divisible only by 1 and itself. The program needs to check each number in the range to see if it is prime. Using an array allows storing prime numbers for easy access and display.

Input-Process-Output:

- **Input:** Start number, End number
- **Process:** Loop through numbers, check if each number is prime, store prime numbers in an array
- **Output:** Display the array of prime numbers

Algorithm:

1. Take start and end numbers from the user.
2. Create an array or ArrayList to store prime numbers.
3. Loop through numbers from start to end.
4. Check if each number is prime.
5. If prime, add it to the array.
6. Display the array after the loop ends.

Source Code:

```
PrimeNumbers.java > ...
1  import java.util.Scanner;
2  import java.util.ArrayList;
3
4  public class PrimeNumbers {
5      public static boolean isPrime(int n) {
6          if (n < 2) return false;
7          for (int i = 2; i <= Math.sqrt(n); i++) {
8              if (n % i == 0) return false;
9          }
10         return true;
11     }
12
13     Run | Debug
14     public static void main(String[] args) {
15         Scanner sc = new Scanner(System.in);
16         System.out.print(s: "Enter starting number: ");
17         int start = sc.nextInt();
18         System.out.print(s: "Enter ending number: ");
19         int end = sc.nextInt();
20
21         ArrayList<Integer> primeNumbers = new ArrayList<>();
22         for (int i = start; i <= end; i++) {
23             if (isPrime(i)) {
24                 primeNumbers.add(i);
25             }
26         }
27
28         System.out.println("Prime numbers in the range: " + primeNumbers);
29         sc.close();
30     }
31 }
```

Output:

```
Enter starting number: 2
Enter ending number: 20
Prime numbers in the range: [2, 3, 5, 7, 11, 13, 17, 19]
PS C:\Users\dell\Downloads\OneDrive\Documents\Sharir's Folde
```

Discussion:

Using `Math.sqrt(n)` to check for factors makes the prime number check efficient. `ArrayList` allows dynamic storage and easy display of prime numbers. This exercise teaches the integration of loops, arrays, and number theory in Java.